2024
算法分析与设计

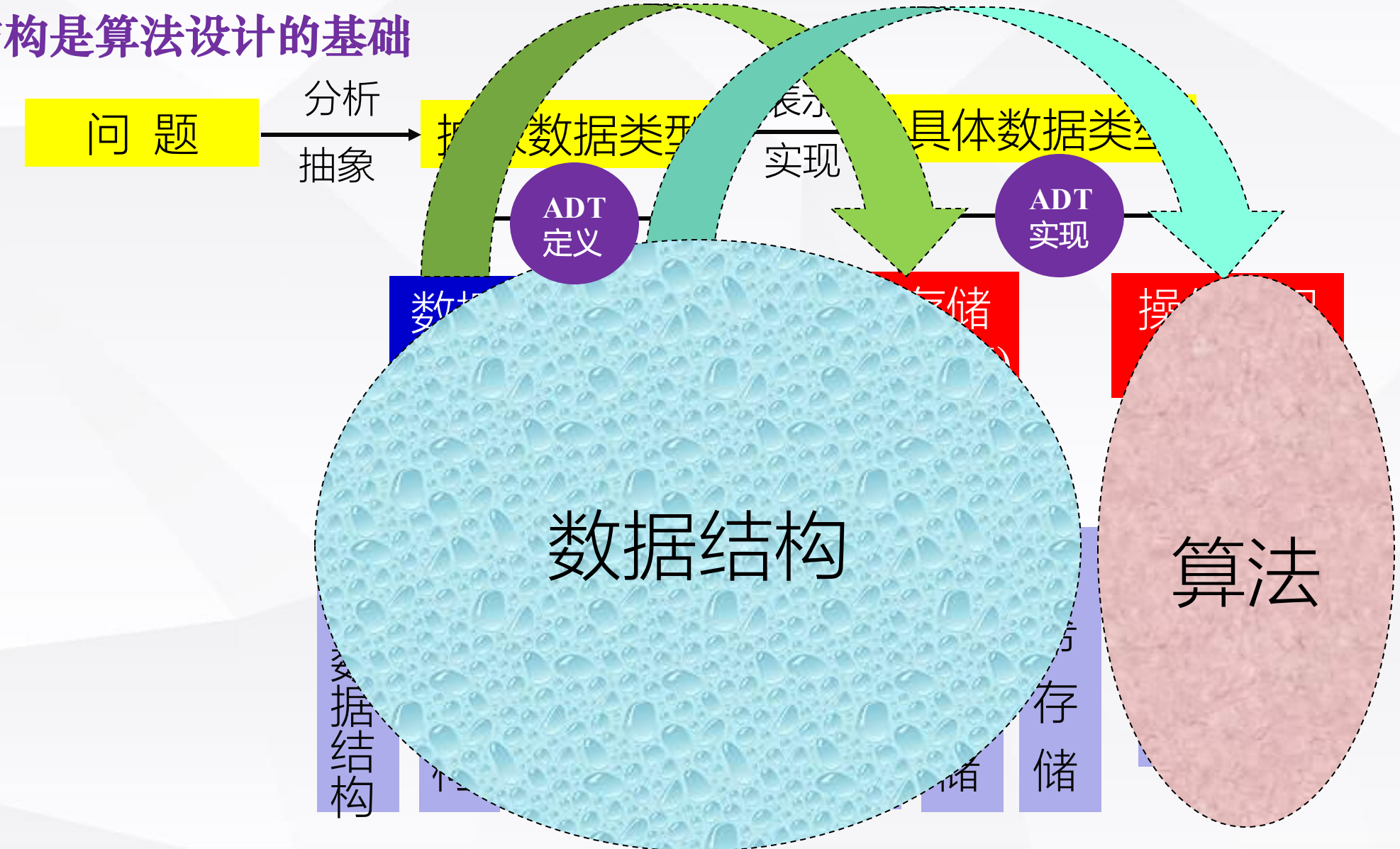
主讲人：孙歧峰

联系电话：13864717907
Q Q号码：274342509
邮　　箱：sunqf@upc.edu.cn

引言—课程的地位

数据结构是算法设计的基础

数据结构与算法设计



引言—课程的地位

万物之灵的人发明了计算机，实现了一种抽象计算模型（基于图灵模型），使人脑得到拓展和延伸。尽管计算机的性能日益提高，但关键因素还在于人类自身。计算机技术的发展改变了人类的各种活动，促进了社会飞速发展，同时也激起了人的欲望需求，即人类的计算机应用需求越来越大。 这样，人的“贪欲”就对IT技术不断提出新的挑战。因为无论怎样，CPU再快也不会无限快，存储再大再便宜也不会无限大和免费！ 如何应对这些挑战?

◆应对策略之一，发展硬件技术：

- 容量更大、更便宜、访问速度更快的存储；
- 速度更快、处理能力更强的CPU；
- 安全可靠、快速的传输介质；

◆应对策略之二，发展软件技术：

- 应用的数据如何存储、如何处理
  (数据结构+算法)
- 如何构建应用（软件工程）
- 如何运行应用（系统工程）

引言—课程的地位

► **算法的重要性，算法的魅力！**

问题1：通过直线*l*上的给定点A，作该直线的垂直线。

怎么做?

公元前2000年，古埃及人给出了著名的算法：



这里的计算工具是什么？ 绳索计算机

引言—课程的地位

► 算法的重要性，算法的魅力！

问题2： 圣经上写着：神6天创造天地万物，第7日安歇。

对于全能的神来说，他完全可以在1天、1秒或者任何他愿意的时间长度里创造出天地万物。那么为什么是6天而不是其他时间长度呢？？

问题3： 三峡工程总工期为17年。

三峡工程是中国、也是世界上最大的水利枢纽工程，是治理和开发长江的关键性骨干工程。兴建三峡工程，是中华民族几代人的夙愿。1994年12月1[illegible]峡工程正式开工。工期17年。那么17年的工期是怎么算出来的？

AOV、AOE
网络分析

引言—课程的地位

► 算法的重要性，算法的魅力！

关于第2个问题是关于数学的算法问题。之所以是6天，因为6是完全数（完美数）！

- 什么是完全数?
- 完全数不止6这一个，但是确实数量很少。直到2009年6月，数学家探索了2600年，还借助了超级计算机等工具，也仅仅找到了47个完全数，6、28、496、8128、33550336…。第47个完全数有“25956377位”，值是$2^{43112608}\times(2^{43112609}-1)$。

公元前3世纪，古希腊著名数学家欧几里得甚至发现了一个计算完全数的公式：如果2^n-1是一个质数，那么，由公式$N=2^{n-1}\times(2^n-1)$算出的数一定是一个完全数。



- **寻找新的完全数与寻找新的质数密切相关。**
- **由欧几里得公式算出的完全数都是偶数！那么，奇数中有没有完全数呢?**
- **哪些数是完全数？如何判断一个数是不是完全数？如何求出完全数?**

引言—课程的地位

- 算法的重要性，算法的魅力！
  - 看似很容易：因式分解+求和。
  - 现实很残酷！

    问题的关键是“因式分解”还没有很好的方法。

    就目前看，既使使用超级计算机来计算，情况也不理想。

这意味着：发明和使用新的计算工具并不能使问题解决，而探究新的方法（因式分解）是核心和关键，而这就是算法要研究的。

这些都昭示着达到完美的难度，神选择6天来创造天地万物也许是因为6是最小的完美数，即创造天地万物对于神来说是轻而易举的一件事情。

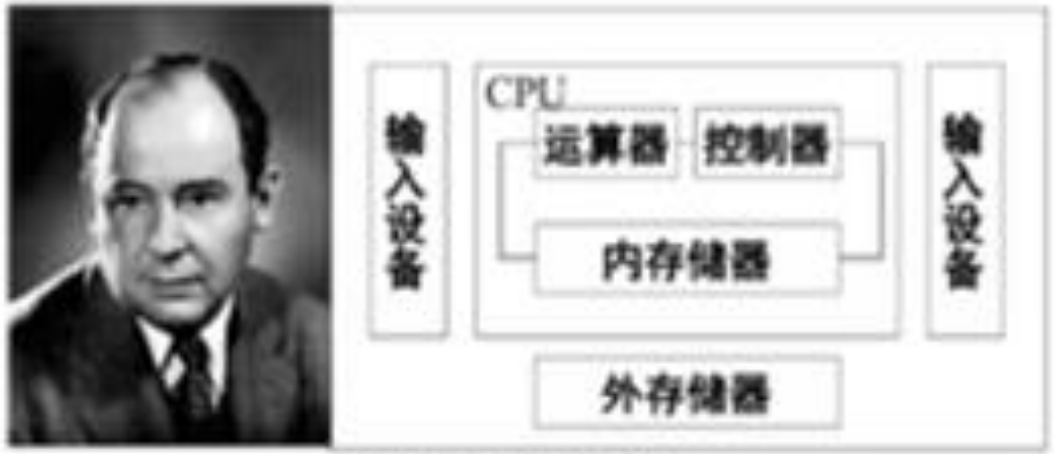
引言—课程的地位

现代算法的由来：算法与计算机的结合

- 1936年，艾伦.图灵提出了图灵机，通过建立通用计算机模型，刻画计算机的计算行为
- 1946年，冯.诺依曼提出了存储程序原理



理论计算机科学与人工智能之父
艾伦·图灵
Alan Turing



现代计算机之父
约翰·冯·诺伊曼
John von Neumann

引言—课程的地位

- 算法无处不在：

算法抽象吗?
算法虚无吗?
算法无聊吗?
看看你身边吧……

去食堂买饭，你会怎么去排队?

你的一天怎么规划和安排?

你用自动售货机时是怎么找零的?

你敲击键盘时，计算机怎么执行命令的?

网络路由是怎么达到的？快、慢?

Google是怎么搜索的？又准又快?

算法总是由相关问题驱动的！

但是算法很难做到完美。比如数据量增大时会怎样？输入变化了会怎样?

一个个新的算法都是为了解决问题已有的算法的不足而产生的，但是新的问题也会有意或无意地引入。

因此，算法永远也不能或不会停止发展。

引言—课程的地位

▶算法是计算机的灵魂：

（1）算法是计算机科学基础的重要主题

- 70年代前，计算机科学基础的主题没有被清楚地认清。
- 70年代，Knuth出版了《The Art of Computer Programming》(三卷），以各种算法研究为主线，确立了算法为计算机科学基础的重要主题，1974年获得图灵奖。
- 70年代后，算法作为计算机科学的核心推动了计算机科学技术的飞速发展。

（2）计算机科学的体系

- 什么是计算？计算机科学的本质：

  ·直观地看，计算一般是指运用事先规定的规则，将一组数值变换为另一(所需的)数值的过程。

- 解决一个计算问题的过程：

  可计算否→能行可计算否→算法设计与分析→用计算机语言实现算法→软件系统

引言—课程的地位

▶算法是计算机的灵魂：

- **可计算理论:**
  - ·计算模型
  - ·可计算问题/不可计算问题
  - ·计算模型的等价性-图灵/Church命题
- **计算复杂性理论（能行否）**
  - ·在给定的计算模型下研究问题的复杂性
    - ·上界　·下界　·平均
    - ·固有复杂性
  - ·复杂性问题的非类：P=NP?
  - ·抽象复杂性研究
- **算法设计和分析**
  - ·可计算问题的算法设计与分析
  - ·设计算法的理论、方法和技术
  - ·分析算法的理论、方法和技术
- **程序设计**
  - . 编码（数据结构实现、算法实现）
  - . 调试
  - . 测试
  - . 运行

引言—课程的地位

“一个受过良好的计算机科学知识训练的人知道如何处理算法，即构造算法、操纵算法、理解算法和分析算法。算法的知识远不是为了编写好的计算程序，它是一种具有一般意义的智能工具，必定有助于对其他学科的理解，不论化学、语言学或者是音乐等” ——**D.E.Knuth**

“算法不仅是计算机学科的一个分支，它更是计算机科学的核心，而且可以毫不夸张地说，它和绝大多数科学、商业和技术都是相关的。”——**David Harel**

- 计算机离不开软件，软件简单说就是程序，而程序的核心是算法。 算法问题是计算机科学研究的核心问题之一！
- **Algorithms＋ Data Structures＝ Programs**

引言—课程的地位

算法与计算复杂性领域图灵奖得主



Donald E. Knuth
1974, USA
算法分析之父



Michael O. Rabin
1976，Israeli
非确定自动机
素数判定随机算法



Dana S. Scott
1976，USA
非确定自动机



Robert W. Floyd
1978, USA
最短路径Floyd算法



Stephen A. Cook
1982, USA
NP完全性



Richard M. Karp
1985, USA
NP完全性与
网络流算法



John Hopcroft
1986, USA
最差情况分析
数据结构与算法



Robert Tarjan
1986, USA
数据结构与图算法



Juris Hartmanis
1993, Latvia
计算复杂性理论



Richard E. Stearns
1993, USA
计算复杂性理论



Manuel Blum
1995, Venezuela
计算复杂性理论



Andrew Yao
2000, China
伪随机数生成
与通信复杂性



Leslie G. Valiant
2010, Hungarian
#P完全性与
计算学习理论

算法与计算复杂性领域图灵奖得主共计13人，占比约18.6%

引言—课程的地位-----改变世界十大算法

01、归并排序(MERGE SORT)、快速排序(QUICK SORT)、堆积排序(HEAP SORT)

02、傅立叶变换、快速傅立叶变换---实现时间域函数与频率域函数之间的相互转化。

03、迪杰斯特拉算法 (Dijkstra's algorithm)

04、RSA非对称加密算法---密钥学领域最牛叉的算法之一，对密钥学和网络安全的贡献

05、哈希算法(Hash Algorithm)一组加密哈希函数

06、整数质因子分解算法(Integer factorization)--将一个合成数分解成不可再分的数因子，加密协议都采用了这个算法。

07、链接分析算法(Link Analysis)---Google的网页排名，Facebook向你发送信息流

08、比例微积分算法--通过"控制回路反馈机制"，减小预设输出信号与真实输出信号间的误差。

09、数据压缩算法--决于应用方向，压缩mp3，JPEG和MPEG-2文件都不一样。

10、随机数生成算法---伪随机数生成算法，在许多领域都有应用，如网络连接，加密技术，安全哈希算法，网络游戏，人工智能，以及问题分析中的条件初始化。

引言—课程的地位



Automate this:how algorithms came to rule our world

“这种颠覆性的黑客范式在20世纪末风行于世界各地,并开启了21世纪:先是一个技术熟练的电脑代码和算法工程师对某一新的领域产生了兴趣,培养自身在该领域的技能,然后应用计算机科学,让代码片段模仿前人的操作,并战胜了无数的公司,破坏了行业标准,击败了行业的老旧势力,从而颠覆了整个行业。”

构建算法模仿超越并最终取代人类,是21世纪最重要的能力。由于具有这种能力的人剧增,工作会逐渐消失,生活会发生巨变,各行各业都会重建规则。这已经发生了,这种趋势还会延续。这一趋势也如历史上的其他趋势一样追逐利益。”

算法正在悄悄的改变世界?

引言—课程的目标

- 求解同一问题通常存在多种算法，其效率不同
- 如何判断哪个算法更高效?
  - ✓ 需要分析比较算法的运行效率
- 如何设计正确高效的算法?
  - ✓ 算法正确性证明
  - ✓ 需要掌握算法设计的方法论



引言—算法设计与人工智能（深度学习）



数据结构与算法设计是深度学习算法优化的基础。

算法大佬访谈　ACM全球冠军访谈　https://www.bilibili.com/video/BV1TC4y1W7wC?p=5

•只要计算机还是图灵原理和冯诺依曼体系结构，数据结构和算法就永远是程序的核心！

引言—参考书目

算法设计与分析基础（C++版）（微课视频版）

（高等学校算法类课程系列教材）

李春葆，陈良臣，喻丹丹 著



- 《Introduction to Algorithms》 Second Edition [美]Thomas H. Cormen, C.E.Leiserson, R.L.Rivest(影印版)
- 《算法导论》[美]Thomas H. Cormen, C.E.Leiserson, R.L.Rivest 潘金贵等译，电子工业出版社 2008年1月
- 《Algorithm Design》[美]Jon Kleinberg ,Eva Tardos 人民邮电出版社，2019年5月
- 《算法设计与分析基础》第2版，第3版影印版已出版 [美]Anany Levitin 潘彦译 清华大学出版社 2007 年1月
- 《算法概论》 [美]Sanjoy Dasgupta , Christos Papadimitriou, Umesh Vazirani 王沛 等译，2008 年7月

麻省理工学院（MIT）公开课：算法导论

集数：24

讲师：*Charles Leiserson&Erik Demaine*

网址：http://v.163.com/special/opencourse/algorithms.html

强烈建议：现在同学们静心读书的太少！至少应读一本原版的经典著作！

引言—课程的主要内容

第一部分：数据结构与算法基础

一般数据结构、高级数据结构

问题及算法、算法的特征

第二部分：数据结构与STL

第三部分：算法分析技术

数学基础

一般分析方法（评估）

第四部分：算法设计策略

基本搜索和遍历（蛮力等）

高级设计方法（分治、贪心、动态规划等）

第五部分：求解困难问题

NP、NPC

* 随机算法、近似算法等

第一部分

数据结构与算法基础

1.1 数据结构与存储结构

数据结构=数据+结构　　记作：

Data_Structure=(D,S)

其中：**Data_Structure**是数据结构的名称

D是数据元素的有限集合（一般为一个数据对象）

S是**D**上关系的有限集

注：这里说的数据元素之间的关系是指元素之间本身固有的逻辑关系，与计算机无关。

又称为：数据的逻辑结构、抽象数据结构

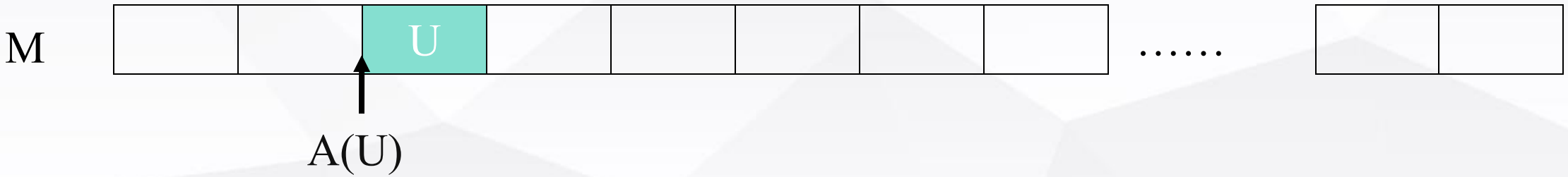
几种逻辑结构？　形式化描述？

1.1 数据结构与存储结构

数据结构的***物理结构***是指逻辑结构的存储镜像(image)。数据结构 DS 的物理结构 P 对应于从 DS 的数据元素到存储区 M (维护着逻辑结构 S) 的一个映射:

$$P:(D, S) \dashrightarrow M$$

存储器抽象模型: 一个存储器 M 是一系列固定大小的存储单元，每个单元 U 有一个唯一的地址 $A(U)$，该地址被连续地编码。每个单元 U 有一个唯一的后继单元 U'=succ(U)。

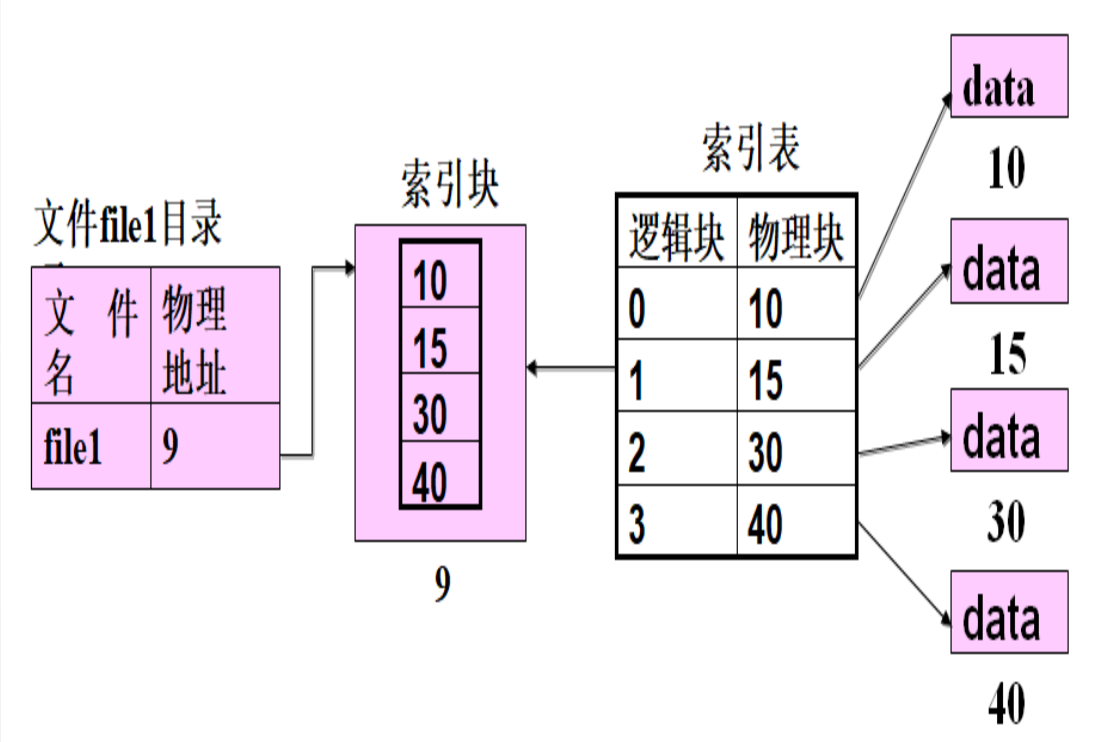


P **的四种基本映射模型:** (各种方式的特点是……)

顺序 (Sequential) 、链接 (Linked) 、索引 (Indexed) 、散列 (Hashing) 映射。

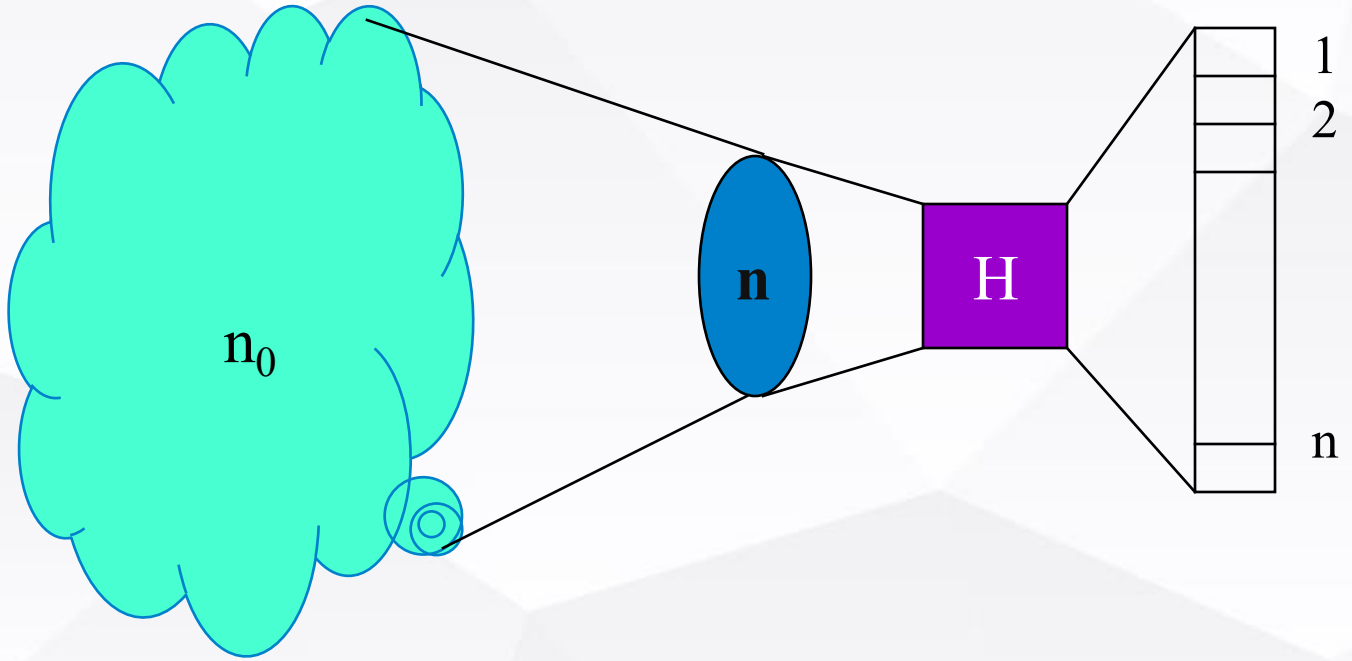
1.1 数据结构与存储结构

索引（Indexed）映射



- 稀疏索引
- 稠密索引

散列（Hashing）映射

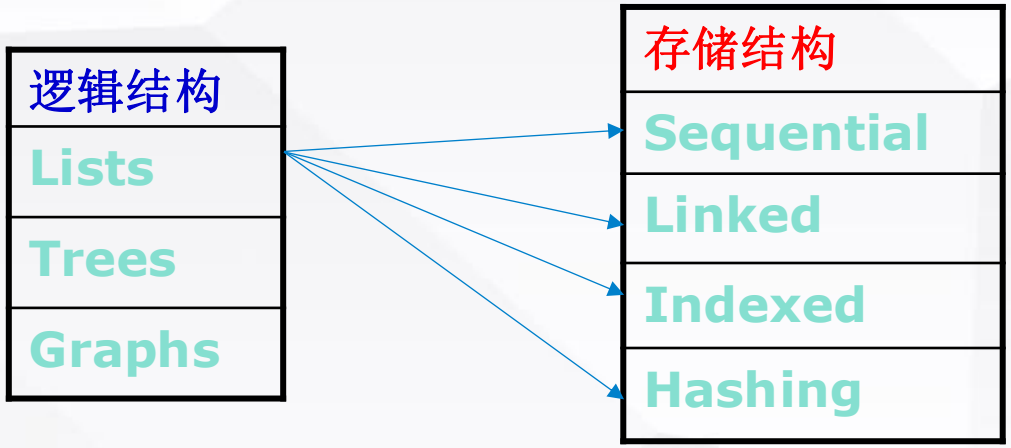


散列存贮的关键：

- 散列函数
- 解决冲突(碰撞)的方法

1.1 数据结构与存储结构

因此，我们至少可以得到4×3=12种可能的物理数据结构：



因此，数据结构主要有三个方面的内容：数据的逻辑结构；数据的物理存储结构；对数据的操作（或算法）。

通常，算法的设计取决于数据的逻辑结构，算法的实现取决于数据的物理存储结构。

1.2 数据结构的评价

好的和坏的DS：如果一个*DS*可以通过某种"线性规则"被转化为***线性的DS***（例如线性表），则称它为好的*DS*。好的*DS*通常对应于好的（高效的）算法。



这是由计算机的计算能力决定的，因为计算机本质上只能存取逻辑连续的内存单元，因此，在*不能线性化的结构逻辑上是不可计算的*。比如对一个图进行操作，要访问图的所有结点，则必须按照某种顺序来依次访问所有节点（要形成一个偏序），必须通过某种方式将图固有的非线性结构转化为线性结构才能对图进行操作。

1.3 基本数据结构

一、线性数据结构

1. 一般的线性表
2. 特殊的线性表

栈

队列，优先队

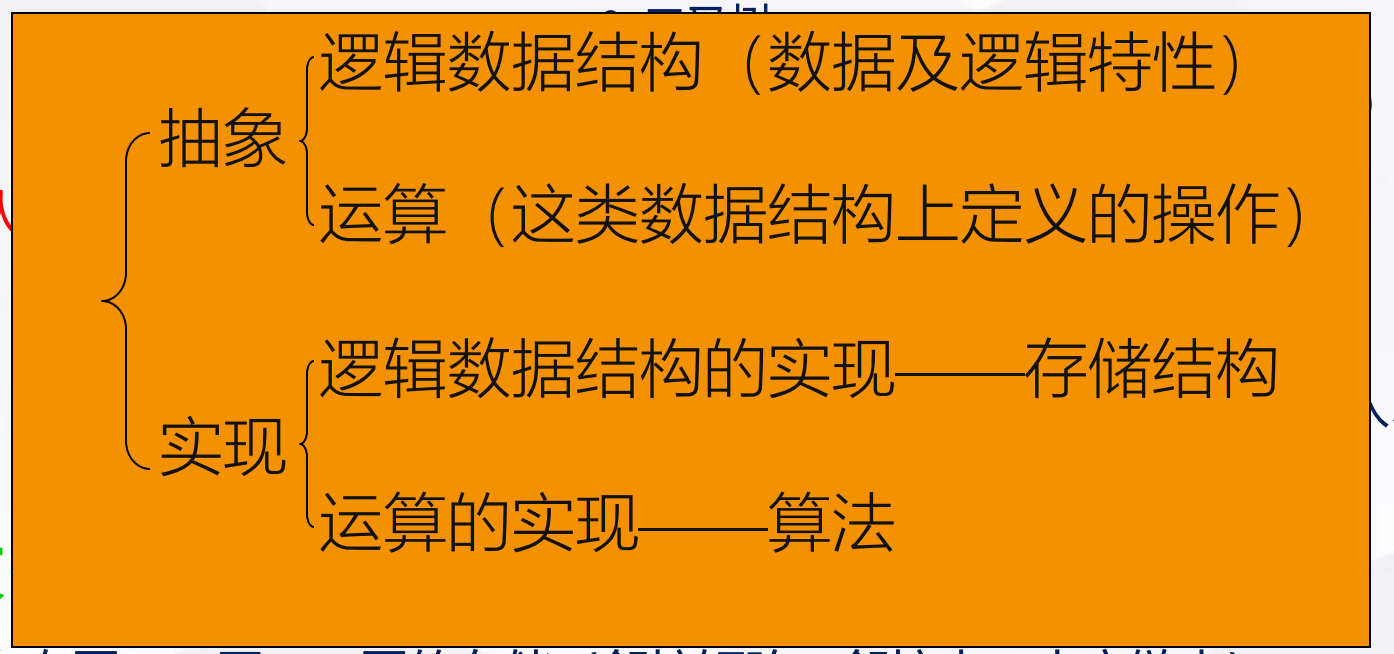
字符串

数组

广义表

二、层次数据结构

1. 树



解压缩

、删除，效率？）

三、网状数

1. 无向图　2. 有向图　3. 网　4. 图的存储（邻接矩阵、邻接表、十字链表）

5. 图的操作—遍历

6. 图的应用：　最小生成树问题　最短路径问题　工程应用（AOV,AOE等）

1.3 基本数据结构

1. 跳跃表（1989年由William Pugh提出）
2. HASH表
3. 矩阵
4. 高级树

    堆（Heap），左式堆，斐波那契堆（Fibonacci Heap）

    B树，B+树，2-3树，红黑树，kd树

    Trie树（字典树）

    Patricia树（Trie的变体）

    伸展树（1985年由Sleator 和 Tarjan提出）

    半伸展（semi-splaying）树

    PR四分树

    ……

算法基础

- 算法的设计策略
- 算法的评价方法
- 算法的实现技术

2.1 问题、算法和程序

1. 问题（Problem）

是一个要求给出解答的一般性提问；　或者说：
是一个需要完成的任务，即对应一组输入有一组相应的输出。

■问题的要素：
实例——描述所有的参量；
询问——陈述解答应当满足的性质；

■数学对问题的定义：
一个问题就是函数，是输入和输出之间的一种映射关系。输入是一个值或一些信息（参量）；

不同的输入可以产生不同的输出，但是相同的输入必须产生相同的输出。

2.1 问题、算法和程序

从数学的角度，可以给出问题的定义

设Input和Output是两个集合。一个问题是一个关系R⊆Input×Output，Input称为问题R的输入集合，Input的每个元素称为R的一个输入，Output称为问题R的输出或结果集合，Output的每个元素称为R的一个结果。

■问题定义了输入和输出的关系。

问题实例:问题R的一个实例是的一个二元组。

例. 排序问题，定义如下：

输入集合 Input={$<a_1, a_2, \ldots, a_n>$ | a_i是整数 }

输出集合 Output={$<b_1, b_2, \ldots, b_n>$ | b_i是整数, $b_1 \le b_2 \le \ldots \le b_n$}

问题SORT={ ($<a_1, \ldots, a_n>$, $<b_1, \ldots, b_n>$) |

$<a_1, \ldots, a_n> \in$ Input, $<b_1, \ldots, b_n> \in$ Output, $\{a_1, \ldots, a_n\}=\{b_1, \ldots, b_n\}$ }

2.1 问题、算法和程序

2. 算法（Algorithm）

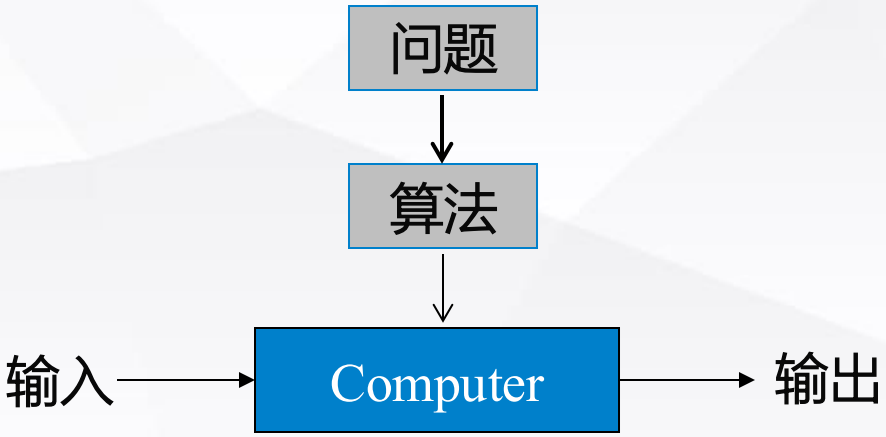
是一个定义良好的计算过程，它以一个或一组值作为输入，并产生输出一个或一组值。

简单说，算法就是解决问题的一种方法或过程，由一系列计算步骤构成（目的是将问题的输入变换为输出）。

$$y = f(x)$$

x 是输入，y 是输出；

算法图示如下：



算法的目的是求解问题。它描述了一个特定的计算过程，用于实现这一输入/输出关系。

2.1 问题、算法和程序

用计算定义算法：“算法是一个满足下列条件的计算”

算法的五要素

- **有穷性/终止性：**有限步内必须停止；
- **确定性：**每一步都是严格定义和确定的动作；
- **能行性：**每一个动作都能够被精确地机械执行；
- **输入：**有一个满足给定约束条件的输入；
- **输出：**满足给定约束条件的结果；

■ 算法与问题：

如果问题的一个算法能应用于该问题的任何一个实例，得到该问题的解，称这个算法解决了这个问题。

注意：一个算法一定是面向一个问题的，而不是仅仅求解一个问题的实例。

2.1 问题、算法和程序

3. 算法的描述方式

- 自然语言
- 图示法（流程图、NS图、PAD图等）
- 算法语言
- 高级语言

4. 算法的评价标准

- 正确性(Correctness)： 穷举证明、谓词演算、数学归纳证明
- 易读性(Readability)： 良好的编程规范
- 健壮性(Robustness)：
- 效率(Efficiency)： 时间,空间

2.1 问题、算法和程序

■ 关于算法的正确性

- **正确算法：**

一个算法是正确的，当且仅当它对于***每一个输入***都最终停止，而且产生正确的输出。

- **不正确算法：**

①不停止(在某个输入上)；

②对所有输入都停止，但对某个输入产生不正确的结果；

- **近似算法：**

①对所有输入都停止；

②产生近似正确的解或产生不多的不正确解；

- **调试程序≠程序正确性证明**

“程序调试只能证明程序有错误，不能证明程序无错误”

2.1 问题、算法和程序

- **算法正确性证明**

（1）证明算法对所有输入都停止；

（2）证明对每个输入都产生正确结果；

— 属（计算机软件）理论的范畴

— 靠数学来解决！

例如：对于循环类算法，证明循环不变式满足3要素性质是证明算法正确性的一种方法。

循环不变式：循环时，循环体内变化的一些核心数据（表达式），它们在循环时保持某种性质不变。

对于循环不变式，必须证明它满足三个性质：

(1) 初始化：它在循环的第一轮迭代开始前是正确的；

(2) 保　持：如果在循环的某次迭代开始前它是正确的，那么，在下次迭代开始之前，它也应该正确；

(3) 终　止：当循环结束时，它是正确的；

类似于数学归纳法

2.2 算法性能

前面提到，要利用计算机求解一个问题，首先确认它是否可计算（可计算否？）；如果可计算，再确定其计算成本是否超出目前系统所提供的计算能力（能行可计算否？）；如果能行可计算，就要设计出正确的算法，并确定其计算的效率。

现在我们关注的是能行可计算的一般性问题，并讨论如何高效率地解决这一层面的问题。为此：

（1）需要确定一种尺度，用以从时间和空间等方面度量算法的计算成本，进而依此尺度对不同算法进行比较和评判；

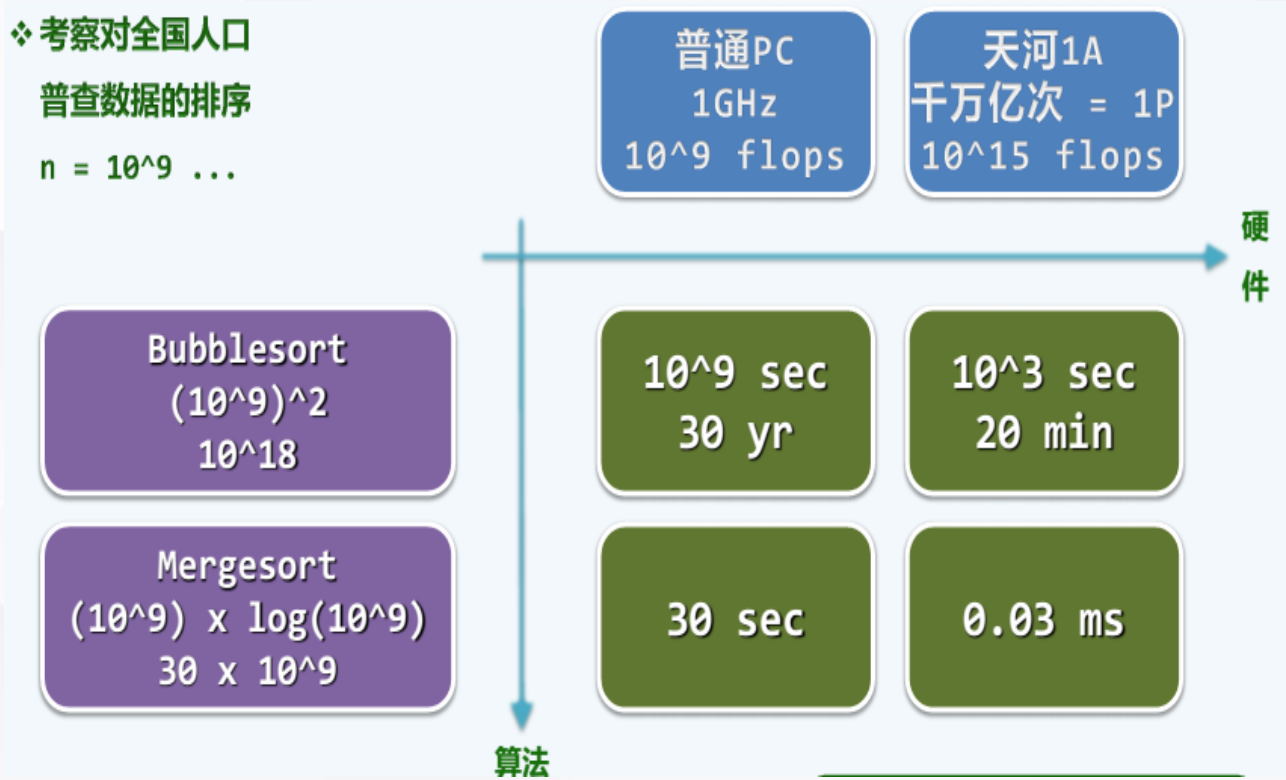
（2）研究和归纳算法设计和实现过程中的一般性规律和技巧；

2.2 算法性能

1. 算法的性能：

指一个算法消耗的资源量（存储量、时间）。又称为算法的复杂性。需要资源越多，复杂性越高；需要资源越少，复杂性越低。

学习和研究算法，是因为“好”的算法它能节约非常多的资源，甚至能完成一些根本不可能完成的 任务。在很多时候，设计优良算法获得的处理的速度提升要远远高于花费金钱去购买新硬件获得的速度提升。特别是在问题规模变大的时候!



2.2 算法性能

2. 算法的空间性能（空间复杂性）

需要空间的大小。在下面场景可能会需要评估空间：

- 在多用户计算机上实现算法；
- 估计算法解决问题的规模；
- 一个问题可能有若干个内存需求各不相同的解决方案；
- 可能需要提前知道是否有足够的内存来运行算法；

3. 算法的空间性能的影响因素

- 指令空间（由机器决定）；
- 数据空间（常量、变量占用空间）；
- 环境栈空间；

2.2 算法性能

指令空间 - 用来存储经过编译之后的程序指令。程序所需的指令空间的大小取决于如下因素：

- 把程序编译成机器代码的编译器；
- 编译时实际采用的编译器选项；
- 目标计算机。

所使用的编译器不同，则产生的机器代码的长度就会有差异；有些编译器带有选项，如优化模式、覆盖模式等等。所取的选项不同，产生机器代码也会不同。

此外，目标计算机的配置也会影响代码的规模。如果计算机具有浮点处理硬件，那么，每个浮点操作可以转化为一条机器指令。否则，必须生成仿真的浮点计算代码，使整个机器代码加长。

一般情况下，指令空间对于所解决的特定问题不够敏感。

2.2 算法性能

数据空间 - 用来存储所有常量和变量的值—数据的存储结构 分成两部分：

- 存储简单量
- 存储复合变量

计算方法：描述算法时，都是用已知的类型的量来描述存储。

单个常量、变量：由机器和编译器规定的类型存储决定

数组变量：所占空间等于数组大小乘以单个数组元素所占的空间。

结构变量：所占空间等于各个成员所占空间的累加；

例如：

double a[100]；所需空间为100×8＝800

int matrix[r][c]；所需空间为 2×r×c

2.2 算法性能

环境栈空间 - 保存函数调用返回时恢复运行所需要的信息。当一个函数被调用时，下面数据将被保存在环境栈中：

- 返回地址；
- 所有局部变量的值、递归函数的传值形式参数的值；
- 所有引用参数以及常量引用参数的定义。

环境栈通常是独立于实例的特征，但递归函数的使用除外！在递归函数情形，实例特征多数情况下会影响环境栈所需空间的数量，因为某些编译器仅为递归函数保存局部变量的值、传值参数的值、引用参数和常量参数的定义。

一般地，一个算法所需要的空间可分为两部分：

固定部分：它独立于实例的特征。主要包括指令空间、简单变量以及定长复合变量所占用的空间、常量所占用的空间等；

可变部分：主要包括复合变量所需的空间（其大小依赖于所解决的具体问题）、动态分配的空间（依赖于实例的特征）、递归栈所需的空间（依赖于实例特征）。

2.2 算法性能

4. 算法的时间性能（时间复杂性）： 需要时间的多少。

在下面场景可能会需要评估时间：

- 有些计算机需要用户提供时间的上限；
- 要求满意的响应时间；
- 一个问题可能有多各不相同的解决方案，时间性能不同；

5. 算法的时间性能的影响因素：

- 机器的运行速度（执行代码的速度）；
- 书写程序的语言；
- 编译产生代码的质量；
- 算法的策略；
- 问题的规模；

2.2 算法性能

6. 算法性能的确定方法：

实验的方法：通过实验测定 (精确测量、事后计算)；

分析的方法：粗略估计（**estimate-评估、**
back of the napkin-餐桌布背面、
back of the envelope calculation-封底估算）

先看一个问题：如果你不查询任何资料，能推算出地球的周长吗？

可以！推算方法是：纽约和洛杉矶的距离大约是3000英里，时差为3小时。3小时是一天24小时的1/8，而24小时正是地球完成一次自转所需要的时间，因此地球的周长必定是把3000英里扩大8倍，即24000英里。这个推算答案是真实可信的，地球实际周长为24902.45英里，误差仅为4%。

这种方法就是"封底估算"，最早由美国著名物理学家恩里科-费米提出，他采用与众不同的思维方式。其中最突出的一项，便是他把某一个问题"化整为零"，划分成许多容易驾驭的小问题，再加以"各个击破"，最后得出正确的结论。

2.2 算法性能

评估是人的重要的基本技能之一，也是工程学的内容之一!

评估的步骤

（1）确定问题的主要参数；

（2）推导出一个与问题参数有关的公式（模型）；

（3）选择参数值，由该公式得出一个评估解；

D.E.Knuth给出的算法评估模型：

- 执行一个操作的耗时；
- 每个操作的执行频度；

因此，对算法的分析，主要是分析和估计出算法中基本操作的次数。一般有两种不同的估算计数方法：

（1）操作计数：首先选择一种或多种关键操作（如加、乘和比较等），然后统计这些操作分别执行了多少次。关键操作对时间复杂性影响最大。

（2）执行步数统计：统计执行步数（step-count）的方法则是统计算法中所有可执行部分的时间开销，即统计每个可执行操作的步数。执行步数也是实例特征的函数。

2.2 算法性能

执行步数计算工作表格

执行步	一次执行所需步数	执行频度	步数
{	0	1	0
float s = 0.0;	1	1	1
for (int i=0; i<n; i++)	1	n+1	n+1
s += a[i];	1	n	n
return s;	1	1	1
}	0	1	0
	总步数		2n+3

不论那种方法，操作次数是与*算法策略(A)、问题规模(n)、算法的输入(I)*有关系的。因此，最终计算出的算法的操作次数的累计和是一个关于***A,n,I***的函数。

这就是目前算法时间复杂性的抽象计量模型： **T=T(n, I, A)**

若特指一个算法（设计出算法后，算法就定了），则A可以省略。 简记为： **T=T(n， I)**

在规模一定的情况下，可以简记为： **T=T(I)**

在不考虑输入分布的情况下，则可以简为： **T=T(n)**

2.2 算法性能

算法空间复杂性的抽象计量模型也类似：

S=S(n, I, A)

S=S(n，I)

S=S(I)

S=S(n)

3.1 复杂性分析的基本概念

1. 算法复杂性

它是度量算法计算难度的一种尺度，反映了算法消耗的资源情况。

算法需要资源越多，其复杂性就越高；反之，

算法需要资源越少，其复杂性就越小。

时间复杂性（Time Complexity）：

如果问题的规模为n，解决这一问题的某算法在算法的输入为I时所需的时间为T(n,I)，T(n，I)称为算法的时间复杂性。

空间复杂性（Space Complexity）：

如果问题的规模为n，解决这一问题的某算法在算法的输入为I时所需的空间为S(n,I)，S(n,I)称为算法的空间复杂性。

算法分析（Algorithm Analysis）:分析算法复杂性的过程；

3.1 复杂性分析的基本概念

算法的复杂性除了与问题的规模有关外，在规模一定的情况下，输入的分布也会影响算法的复杂性。从而就有了最好、最坏、平均复杂性。

算法的最好时间复杂性：

$T_{min}(n)=\min T(n,I)=T(n,I^*)$ $\quad I\in D_n$

注：输入规模的概念和具体问题有关，对有些问题来说，最自然的度量标准是输入中的元素个数（例如，排序）；对另一些问题其规模的最佳度量是输入数在二进制表示下的位数（例如整数相乘）；还有的问题需要用两个数表示输入规模（例如图的顶点数和边数）。

$T_{avg}(n)=\sum_{I\in D_n} P(I)\,T(n,I)$ $\quad I\in D_n$

D_n是规模为n的合法输入的集合,p(I)是输入I出现的概率。

对算法空间复杂性可以同样定义。

实践证明，可操作性最好且最有实际价值的是最坏复杂性！

3.1 复杂性分析的基本概念

对于一个算法而言，在不同的执行时间内，它占用的内存空间量不一定相等，也就是说，占用空间量y是时间x的函数，即y=f(x)。

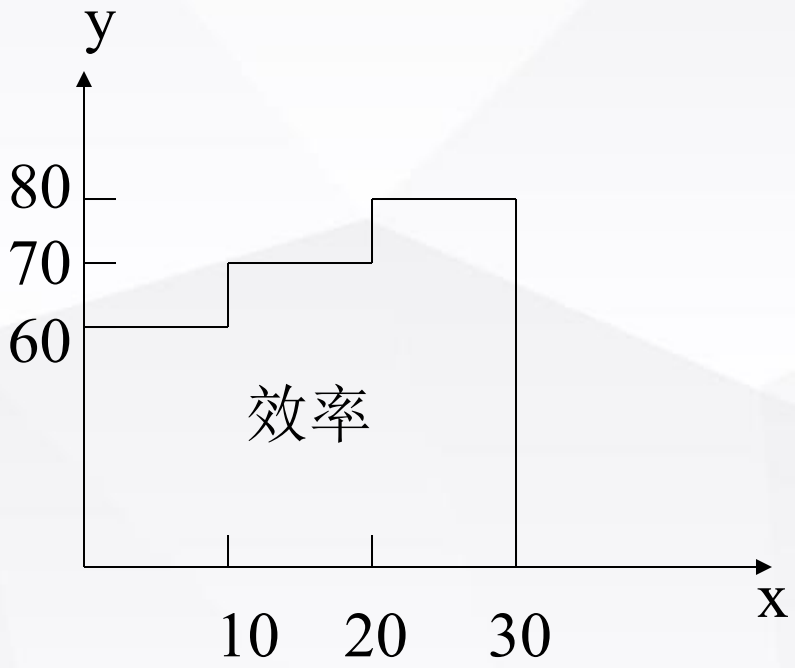
定义：时空积分= $\int_0^t f(x)dx$

其中t是算法的运行时间。

时空积可以综合评价算法的时间、空间性能。

例如：一个算法的执行时间为30秒，前10秒算法占用60个字节，第2个10秒占用70个字节，最后10秒占用80个字节，则：时空积如图。

60*10+70*10+80*10=2100(字节秒)



3.1 复杂性分析的基本概念

2. 问题的时间（空间）复杂性：

假设问题有多个算法，所有算法的集合表示为Γ，其中A是其中的一个算法，即$A \in \Gamma$，它的时间（空间）复杂性表示为$T_A(n)$，问题的时间（空间）复杂性定义为：$\mathrm{Min}(T_A(n))$。 $A \in \Gamma$

即最优算法的时间（空间）复杂性为该问题的时间复杂性。

3.2 复杂性渐近态

一个关注点， 算法性能的增长率！

算法的增长率(growth rate)：是指当问题规模增长时，算法代价的增长速率。具体就体现在两个函数的变化趋势上。

T(n)=f(n)

S(n)=g(n)

简单说，就是看看这个算法的资源消耗的增长有多快，增长快的（阶高），其资源消耗就大，复杂性就高！ ——*这就是复杂性渐近态的思想*

3.2 复杂性渐近态

1. 复杂性的渐近态

对算法，精确计算其增长率有时很难，如果能粗略比较出哪个更快或更慢就可以了。于是在关注增长率的基础上提出渐近估计的概念。即规模变大时，趋势渐近于什么。 “和谁差不多一样快”

2. 渐近复杂性分析（Asymptotic Algorithm Analysis）

确切说：渐近分析是指当输入规模很大，或者说大到一定程度时对算法的研究和分析。（从微积分意义上说是达到极限时）。 这样得到的就是算法的渐近复杂性。

例如： $T_1(n)=3n^2+4n\log n$

$T_2(n)=2n^2$

在n很大后，两个函数变化趋势接近一致。可以说$n\to\infty$时，$T_1(n)$渐近于$T_2(n)$，可以用简单的$T_2(n)$来表示算法的复杂性。当然，更简单的函数是n^2。 显然，应该用最简单的函数来代表增长率一样的这一类。

3.2 复杂性渐近态

因此，渐近复杂性分析只关心度量函数的阶就够了，因为按照渐近的思想，度量函数中的常数因子对算法复杂性的影响不敏感。

这样，我们在考虑增长率时就不用过多地专注于它的精确，而是专注于它的“数量级”或者“阶！

所以，渐近复杂性分析主要专注于问题的“规模”、“基本的操作”和增长率。

“规模”和“基本操作”都是模糊的概念！

“规模”：一般是指输入量的数目。

“基本操作”：完成该操作所需时间与操作数的具体取值无关 。通常我们都是假设各个基本操作执行1次所需的时间都一样，取一个单位时间。

要特别注意渐近的含义，即并不是任何情况都可以忽略常数！！当解决问题的规模n很小时，系数就会起到举足轻重的作用。

3.3 渐近符号（简单了解）

2．一些标准记号和通用函数

（1）上取整(Floors)下取整(Ceilings)

对于任意实数x，小于或等于x的最大整数表示为$\lfloor x \rfloor$，大于或等于x的最小整数表示为$\lceil x \rceil$。对于所有实数x，$x-1<\lfloor x \rfloor \le x \le \lceil x \rceil < x+1$

对于任何整数n，$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$

对任意实数$n \ge 0$和整数$a,b>0$，$\lceil \lceil n/a \rceil / b \rceil = \lceil n/ab \rceil$

$$\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor$$

$$\lceil a/b \rceil \le (a+(b-1))/b$$

$$\lfloor a/b \rfloor \ge (a-(b-1))/b$$

下取整函数$f(x)=\lfloor x \rfloor$是单调递增的，上取整函数$f(x)=\lceil x \rceil$也是单调递增的。

3.3 渐近符号

(2) 取模运算 (Modular arithmetic)

对于任意整数a和任意正整数n，a mod n的值即是a/n的余数： $a \bmod n = a - \lfloor a/n \rfloor n$

同余：如果$(a \bmod n) = (b \bmod n)$，称在模n时，a等于b，即a和b被n除时余数相同。

记作 $a \equiv b \pmod n$ **$a \equiv b \pmod n$当且仅当n是b-a的一个约数。**

(3) 多项式

给定一个正整数 d， n 的 d 次多项式是具有如下形式的函数p(n)：

$$p(n)=\sum_{i=0}^{d} a_i n^i$$

其中常数$a_0, a_1, \ldots, a_d$是多项式的系数，且$a_d \neq 0$。

一个多项式是渐近正的，当且仅当$a_d>0$。

. 对于一个d次的渐近正多项式p(n)，有$p(n)=\Theta(n^d)$。

. 对于任意实常数$a \geq 0$，函数n^a单调递增；对于任意实常数$a \leq 0$， 函数n^a单调递减；

. 对某个常数k有$f(n)=O(n^k)$，则称函数f(n)有多项式界；

3.3 渐近符号

(4) 指数式

对于任意实数a>0，m和n，有下列恒等式：

$a^0=1$ ， $a^1=a$ ， $a^{-1}=1/a$

$(a^m)^n=a^{mn}$， $(a^m)^n=(a^n)^m$， $a^m a^n=a^{m+n}$

对任意n和a>1，函数a^n关于n单调递增；对任意实数a和b，且a>1，有：$\lim_{n\to\infty} \frac{n^b}{a^n}=0$

因此，根据前面的定义可知 $n^b=o(a^n)$ 即：**任何底大于1的指数函数比任何多项式函数增长的更快！**

用e表示2.71828…，即自然对数函数的底，

对任意实数x

$$e^x=1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots=\sum_{i=0}^{\infty}\frac{x^i}{i!}$$

对任意实数x，有不等式：

$e^x \geq 1+x$ 只有x=0时等号才成立；

当 $|x|\leq 1$时，有近似式：

$1+x\leq e^x\leq 1+x+x^2$

当x→0时，用1+x来近似e^x的效果相当好：

$e^x=1+x+\Theta(x^2)$

对任意的x，有：

$$\lim_{n\to\infty}\left(1+\frac{x}{n}\right)^n=e^x$$

3.3 渐近符号

（5）对数

一般约定：$\lg n=\log_2 n$

$\ln n=\log_e n$

$\lg kn=(\lg n)k$

$\lg\lg n=\lg(\lg n))$

$\lg n+k=(\lg n)+k$

函数来$\log_b n$在常数b大于1时，关于n>0严格递增。

$a=b^{\log_b a}$

$\log_c(ab)=\log_c a+\log_c b$

$\log_b a^n = n\log_b a$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$\log_b(1/a)= -\log_b a$

$$\log_b a = \frac{1}{\log_a b}$$

$a^{\log_b c}=c^{\log_b a}$

3.3 渐近符号

改变一个对数的底只是把对数的值改变了一个常数倍，所以当不在意这些常数因子时常采用“lgn”记号，就象O记号一样。一般认为对数的底取2最为自然，因为很多算法和数据结构都涉及到对问题的二分。

当|x|<1时，ln(1+x)的一个简单级数展开为：

$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\frac{x^5}{5}-\cdots\cdots$$

当x>-1时，还有以下不等式成立：

$$\frac{x}{1+x}\leq \ln(1+x)\leq x$$

只有x=0时等号才成立

3.3 渐近符号

如果对常数k，函数$f(n)=O(\lg^k n)$，则称f(n)是*多项对数*有界的。

由前面我们已知：

$$\lim_{n\to\infty} \frac{n^b}{a^n}=0$$

于是用lg n 代替n，用2^a代替a，于是可以将多项式增长率和多项对数增长率联系起来：

$$\lim_{n\to\infty} \frac{\lg^b n}{(2^a)^{\lg n}}=\lim_{n\to\infty} \frac{\lg^b n}{n^a}=0$$

因此对于任意常数a>0，有：$\lg^b n=o(n^a)$

即任意正的多项式函数都比多项对数函数增长得快！

3.3 渐近符号

(6) 阶乘

阶乘函数的一个弱上界是 $n! \leq n^n$ 。

斯特林近似公式：

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1+\Theta\left(\frac{1}{n}\right)\right)$$

给出了一个更紧确的上界和下界。其中e是自然对数的底。

有用结论：

$n! = o(n^n)$

$n! = \omega(2^n)$

$\lg(n!) = \Theta(n\lg n)$

3.4 复杂性分析的内涵

1. 算法复杂性的实质

算法复杂性的实质：问题规模变化与资源消耗的关系！！

同一个问题，用不同的算法解决，规模相同，但是占用的存储资源量、花费的时间会不同！

渐进复杂性分析的重要性

很多人认为：随着计算机技术的发展，低效算法无所谓了！其实不然，“坏”算法终究是“坏”算法，不因为计算机速度的提高它就成了“好”算法！

对低效算法（一般是超线性的）来说，计算机本身性能的提高不能带来求解问题规模的增益。即：计算机速度提高了，我们希望在原来的时间内解决问题的规模变大，或希望解决原来相同规模的问题时时间相应的变短，但结果可能不是你想象的那样好，这与你使用的算法有关！！

3.5 复杂度分析的常见形式

复杂度分析的主要方法

迭代：级数求和

递归：递归跟踪 + 递推方程

猜测 + 验证

常见的级数：

❖ 算数级数：与末项平方同阶

$$T(n) = 1 + 2 + \ldots + n = n(n+1)/2 = O(n^2)$$

❖ 幂方级数：比幂次高出一阶：

$$\sum_{k=0}^{n} k^d \approx \int_0^n x^d\,dx = \frac{1}{d+1}x^{d+1}\Big|_0^n = \frac{1}{d+1}n^{d+1} = O(n^{d+1})$$

$$T_2(n) = 1^2 + 2^2 + 3^2 + \ldots + n^2 = n(n+1)(2n+1)/6 = O(n^3)$$

$$T_3(n) = 1^3 + 2^3 + 3^3 + \ldots + n^3 = n^2(n+1)^2/4 = O(n^4)$$

$$T_4(n) = 1^4 + 2^4 + 3^4 + \ldots + n^4 = n(n+1)(2n+1)(3n^2+3n-1)/30 = O(n^5)$$

...

3.5 复杂度分析的常见形式

❖ 几何级数（a > 1）：与末项同阶

$$T_a(n) = a^0 + a^1 + \dots + a^n = (a^{n+1} - 1)/(a - 1) = \mathcal{O}(a^n)$$

$$1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1 = \mathcal{O}(2^{n+1}) = \mathcal{O}(2^n)$$

❖ 收敛级数

$$1/1/2 + 1/2/3 + 1/3/4 + \dots + 1/(n-1)/n = 1 - 1/n = \mathcal{O}(1)$$

$$1 + 1/2^2 + \dots + 1/n^2 < 1 + 1/2^2 + \dots = \pi^2/6 = \mathcal{O}(1)$$

$$1/3 + 1/7 + 1/8 + 1/15 + 1/24 + 1/26 + 1/31 + 1/35 + \dots = 1 = \mathcal{O}(1)$$

❖ 可能未必收敛，然而长度有限

$$h(n) = 1 + 1/2 + 1/3 + \dots + 1/n = \Theta(\log n)$$ //调和级数

$$\log 1 + \log 2 + \log 3 + \dots + \log n = \log(n!) = \Theta(n \log n)$$ //对数级数

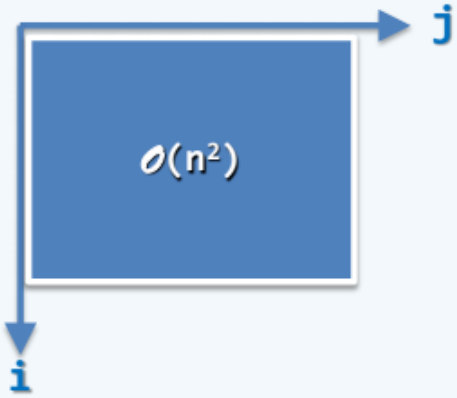
3.5 复杂度分析的常见形式

循环 vs. 级数

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
     O1Operation(i, j);
```

算术级数：

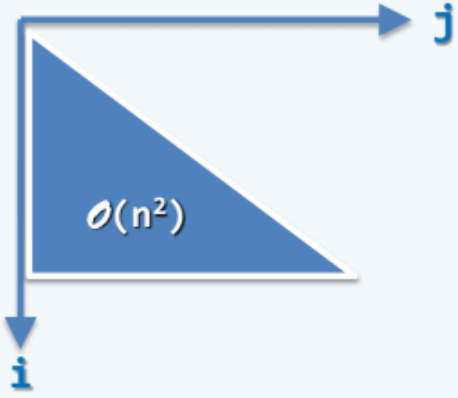
$$\sum_{i=0}^{n-1} n = n + n + \ ... + n = n * n = O(n^2)$$




```
for (int i = 0; i < n; i++)
  for (int j = 0; j < i; j++)
     O1Operation(i, j);
```

算术级数：

$$\sum_{i=0}^{n-1} i = 0 + 1 + \ ... + (n-1) = \frac{n(n-1)}{2} = O(n^2)$$



3.5 复杂度分析的常见形式

循环 vs. 级数

- `for (int i = 0; i < n; i++)`

  `for (int j = 0; j < i; j += 2015)`

  `O1Operation(i, j);`

  算术级数： ...

- `for (int i = 1; i < n; i <<= 1)`

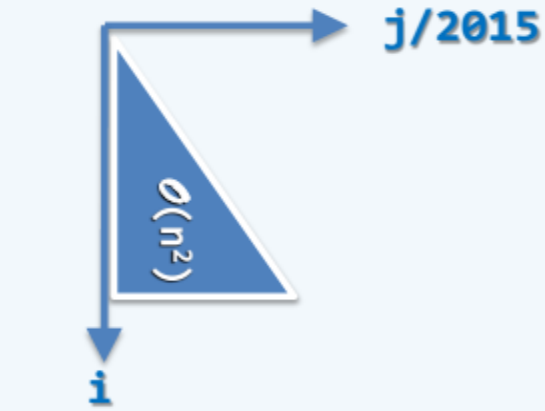
  `for (int j = 0; j < i; j++)`

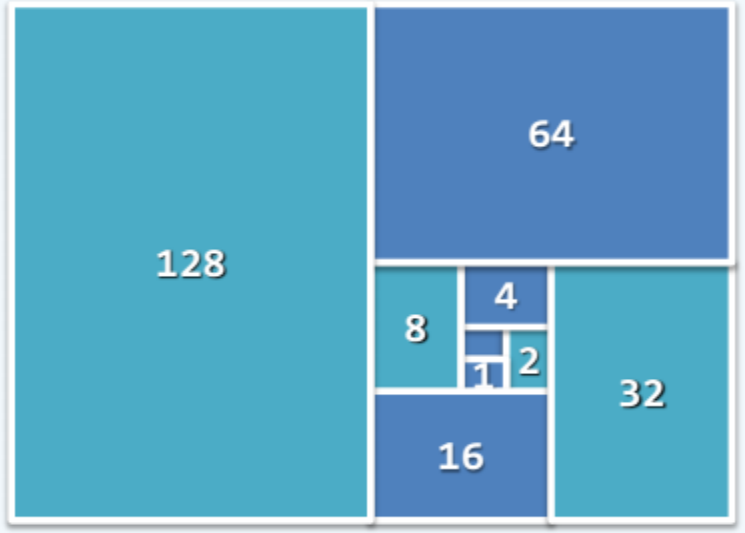
  `O1Operation(i, j);`

  几何级数：$1 + 2 + 4 + \ldots + 2^{\lfloor \log_2(n-1) \rfloor}$

  $= \sum_{k=0}^{\lfloor \log_2(n-1) \rfloor} 2^k$ (let k = $\log_2 i$)

  $= 2^{\lceil \log_2 n \rceil} - 1 = O(n)$





3.5 复杂度分析的常见形式

循环 vs. 级数

```
for (int i = 0; i <= n; i++)
  for (int j = 1; j < i; j += j)
    O1Operation(i, j);
```

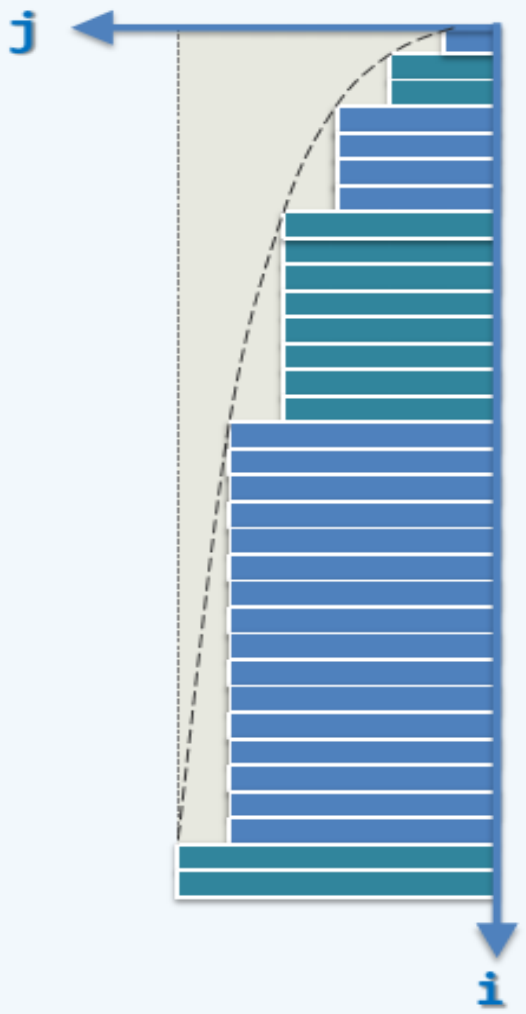
几何级数：$\sum_{k=0}^{n} \lceil \log_2 i \rceil = O(n \log n)$

(i = 0, 1, 2, 3~4, 5~8, 9~16, ...)

$= 0 + 0 + 1 + 2*2 + 3*4 + 4*8 + \ldots$

$= \Sigma_{k=0..\log n}(k * 2^{k-1})$

$= O(\log n * 2^{\log n})$



谢谢大家